Architecture

Living Application Knowledge Platform

Knowledge Transfer Document

29 September 2026

Table of Contents

- Layers and components
- Target system architecture
- Forge concepts as used here
  - asUser versus asApp
- Target data flow
- External services
- Design decisions
- Constraints and limits

Layers and components

Layer	Components	Runs where
Presentation	UI Kit panels and macro, two Custom UI apps	Browser (Atlassian iframe)
API (resolvers)	`resolver.define(...)` in `src/index.js`	Forge function
Event handlers	`onConfluencePage` in `src/triggers.js`	Forge function
Domain libraries	`src/lib/*` (Confluence, Jira, ADF, sessions, insights)	Forge function
Agent seam	`src/lib/agentClient.js`	Forge function
Persistence	Forge KVS through `@forge/kvs`	Forge platform
External services	Agent server, `local-context-service`, `local-flow-service`	Outside Atlassian
Atlassian APIs	Confluence REST v2 and search, Jira REST v3	Atlassian Cloud

Target system architecture

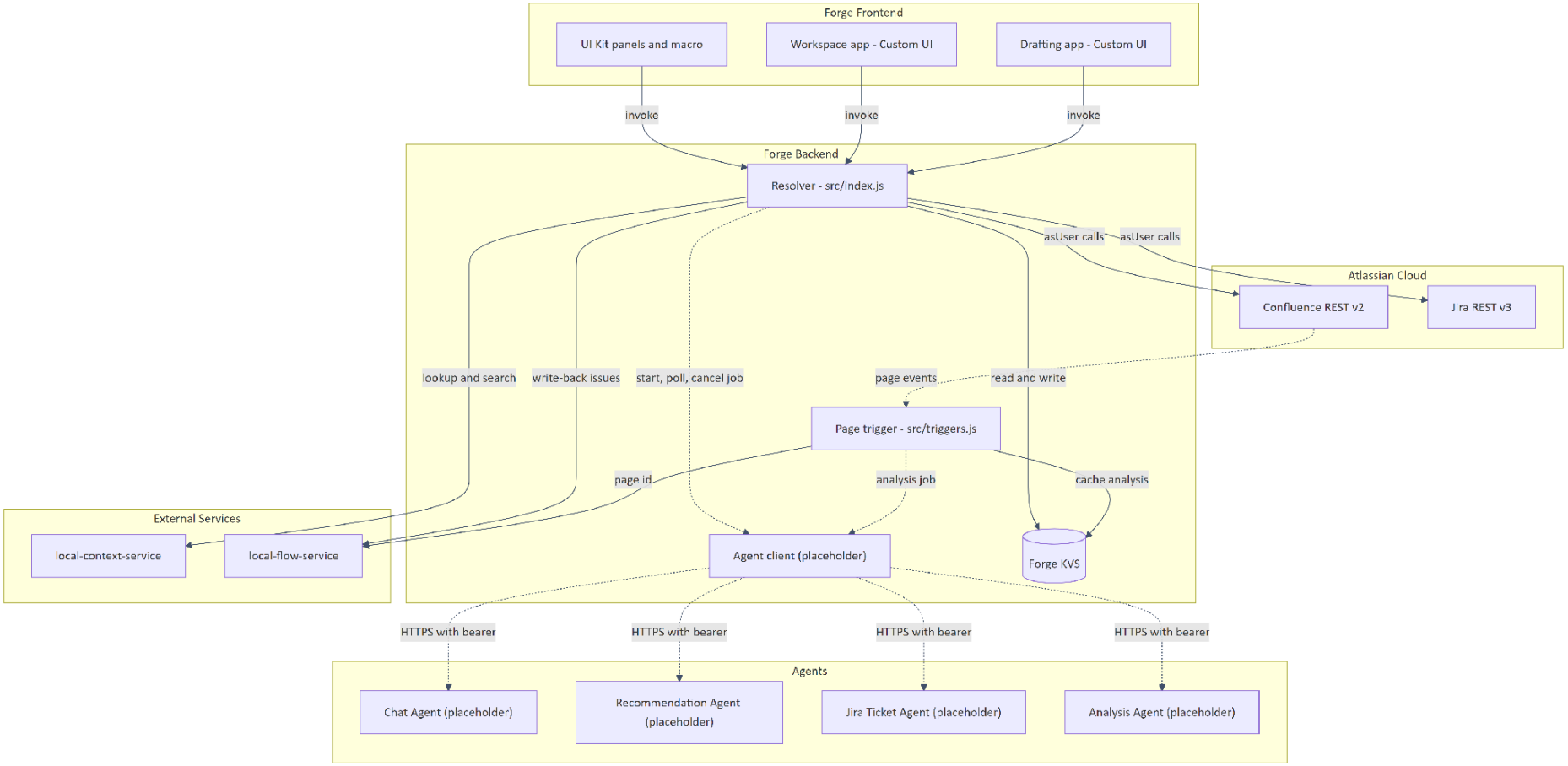


Figure 1: *Flux-Spine target system architecture*

Solid edges are live today. Dashed edges are planned or not yet live: the agent client currently answers locally instead of opening a connection, and the chat agent has no caller yet.

Forge concepts as used here

Concept	How this app uses it
Modules	Issue panel, byline item, admin page, global settings, macro, content action, two global pages, one trigger (see Manifest)
Function	Two: `resolver` (all UI calls) and `confluenceTrigger`
Resolver	One `Resolver` instance in `src/index.js` with 32 named definitions. `export const handler = resolver.getDefinitions()`
Bridge	UI calls `invoke` to reach resolvers; `router.open` opens the drafting app in a new tab; `view.getContext` and `view.resize`
Storage	KVS only (`kvs.get`, `kvs.set`, `kvs.delete`). No Forge SQL, no custom entities, no queries or indexes
Events	Product events `avi:confluence:created:page` and `avi:confluence:updated:page`
Egress	`fetch` from `@forge/api` to hosts in `permissions.external.fetch.backend`
Auth	Atlassian APIs are called `asUser()` almost everywhere; see below

`asUser` versus `asApp`

`api.asUser()` sends the request with the current user's permissions and attributes changes to them. `api.asApp()` uses the app's identity. This code base uses `asUser()` for **every** Confluence and Jira call. The reasons recorded in comments: a user who cannot see or edit something cannot do so through the app, and page history and Jira history show the real person. `asApp()` is not used anywhere.

Target data flow

1. A PM opens the workspace or drafting app (Custom UI) or a panel (UI Kit).
2. The UI calls a resolver with `invoke`.
3. The resolver reads Atlassian data as the user, assembles a payload, and asks the agent client to start a job. It returns a job id immediately.
4. The UI polls a resolver (`getJobResult`) which asks the agent client for one status check.
5. When the agent returns a proposal, the UI shows a diff. The PM edits and approves.
6. Resolvers save a draft page, publish it, then create Jira issues, all as the user.
7. The created issues are posted to `local-flow-service` so the knowledge bundle stays current.
8. Confluence page events fire the trigger, which forwards to the flow service and requests analysis, caching the result for the macro.

See Application flow for sequence diagrams.

External services

Service	Direction	Purpose	Status
Agent server	Forge to agent	Four job-based agent endpoints	Status: Placeholder (planned)
`local-context-service`	Forge to service	Lookup, search, recommend, health	Status: Implemented (client side)
`local-flow-service`	Forge to service	Bundle write from Confluence and Jira events	Status: Implemented (client side)

The two `local-*` services sit behind one gateway host that splits by path prefix (`/context`, `/flow`). `deriveFlowBaseUrl` in `src/lib/bundleWriteBack.js` derives the flow URL from the configured context URL.

Design decisions

Decision	Reason (from code comments)
Job and poll, polling in the browser	A resolver that waits for an agent would be capped by the function time limit
One agent seam file	Switching from local stand-in to live server should be a one-line change with no manifest edit
Payload assembled in the resolver	Page text is already server-side; avoids sending large text to the browser and back
Text capped from the middle	Keeps a page's opening and closing (`capText` in `src/lib/confluence.js`)
Draft is a real Confluence page	Renders exactly like a page; macros, comments, sharing all work
Publish edits by re-applying changes to the live page	Avoids reverting someone else's edit to a different section
Section hash (`expectedHash`) before overwrite	A changed section is reported as blocked, not overwritten
Create Jira issues one phase per call	Each call must finish inside the resolver time limit; progress is saved after each phase
Macro never calls an agent	Macro renders on every page view; analysis is done at write time and cached
Two apps, handover through KVS	Reading query parameters in a global page is not documented; a KVS key is used instead
Bookkeeping in a separate `finishTicketRun` call	Keeps the creation response inside the time limit

Constraints and limits

Figures in the first table come from this repo's code and comments. Platform limits in the second table are general Forge facts to verify against current Atlassian documentation before relying on them.

Constraint	Value	Where
Resolver time budget assumed by code	25 seconds	comments in `src/index.js`, `src/lib/jiraCreate.js`
One Jira creation phase budget	16 seconds	`PHASE_BUDGET_MS` in `src/lib/jiraCreate.js`
Jira bulk create chunk	10 issues	`BULK_LIMIT` in `src/lib/jiraCreate.js`
Agent request timeout	20 seconds per call	`REQUEST_TIMEOUT_MS` in `src/lib/agentClient.js`
Context service request timeout	10 seconds	`src/lib/contextService.js`
Flow service write-back	4 seconds per call, 12 seconds total	`src/lib/bundleWriteBack.js`
Trigger forward timeout	15 seconds	`src/triggers.js`
Page text sent to agent	24,000 characters; parent 8,000	`src/index.js`
Sessions kept per account	25	`MAX_SESSIONS` in `src/lib/sessions.js`
Fan-out re-analysis	5 pages per edit	`fanOutTargets` in `src/lib/insights.js`
Client wait for drafting job	16 minutes	`draft/App.jsx`
Client wait for ticket job	5 minutes	`draft/jira/JiraStep.jsx`

Platform limit (verify)	Relevance
Function invocation timeout	Trigger work (flow call plus analysis plus fan-out) may exceed the default; see Open questions
Payload size limits for invoke and fetch	Large pages and reports travel in payloads
KVS value size limit	`insights:<pageId>` stores full page text
Confluence and Jira API rate limits	Fan-out, bulk create and per-issue reads

Continue with Manifest and modules.